

ARM NEON vs. Scalar Dot Product - Raspberry Pi

Emmanuel Cano

Objective

- Use ARM NEON instructions to improve the C++ dot product program.
- Measured scalar (compiled -O3) vs manual NEON intrinsics (compiled -O3)
- Goal: explain assembly similarities or differences, and compare performance.

Methodology

System Information

Table 1. System Configuration

Component	Raspberry Pi (Specification)
Operating System	Debian GNU/Linux 12 (bookworm)
Processor	Cortex-A76
Architecture	aarch64
Compiler	g++ 12.2.0
C++ Standard	C++ 17

The Raspberry Pi platform was selected due to its native support for ARM NEON SIMD (Single Instruction, Multiple Data) instructions.

Implementations Evaluated

Two dot product implementations were evaluated in this study:

- Scalar Implementation:
A conventional C++ implementation that computes the dot product using a scalar loop and floating-point multiply-accumulate operations.
- ARM NEON Implementation:
A manually optimized version using ARM NEON intrinsics to exploit SIMD parallelism by processing multiple floating-point values simultaneously.

Both implementations compute the dot product of two single-precision floating-point arrays and produce identical numerical results.

Benchmark Program

A dedicated benchmark program (benchmark.cpp) was written in C++17 to measure the execution time of both dot product implementations. Each function was invoked repeatedly within the same executable to ensure identical runtime conditions, including memory layout, cache behavior, and calling context.

Timing measurements were obtained using `std::chrono::high_resolution_clock`. Multiple iterations were executed for each implementation to reduce the effects of measurement noise and short-term system variability.

Compilation and Optimization

All source files were compiled using the `-O3` optimization level and the C++ standard. This optimization level enables aggressive compiler optimizations, including loop unrolling, instruction scheduling, and automatic SIMD vectorization.

No additional compiler flags were used to explicitly enable or disable vectorization. As a result, the scalar implementation was automatically vectorized by the compiler when possible, allowing a direct comparison between compiler-generated SIMD code and manually written NEON intrinsics.

Experimental Setup

Both dot product functions were benchmarked within the same execution environment to ensure fairness. Identical input arrays were used for all runs, and data was initialized prior to timing measurements to avoid including setup costs in the results. Care was taken to ensure that only the computation itself was measured.

Assembly Inspection

In addition to runtime measurements, the generated assembly code was examined to analyze instruction selection, loop structure, and vectorization behavior. Assembly output was obtained from both standalone compilation units and from the benchmark executable. The `.s` files were obtained using `'g++ -S'` along with the `-O3` optimization flag.

This comparison allowed differences between baseline scalar code, compiler auto-vectorized code, and manually optimized NEON code to be identified and explained in terms of instruction usage and reduction strategy.

Baseline Scalar Implementation (Non-Vectorized) (dot_product_scalar.cpp)

```
.global      _Z18dot_product_scalarPKfS0_i
.type       _Z18dot_product_scalarPKfS0_i, %function
_Z18dot_product_scalarPKfS0_i:
.LFB1780:
.cfi_startproc
movi        v0.2s, #0
cmp         w2, 0
ble         .L1
sbfiz       x3, x2, 2, 32
mov         x2, 0
.p2align    3,,7
.L3:
ldr         s2, [x0, x2]
ldr         s1, [x1, x2]
add         x2, x2, 4
fmadd       s0, s2, s1, s0
cmp         x3, x2
bne         .L3
.L1:
ret
.cfi_endproc
.LFE1780:
.size       _Z18dot_product_scalarPKfS0_i, .-_Z18dot_product_scalarPKfS0_i
```

- `movi v0.2s, #0` : zero accumulator s0
- `cmp w2, 0` : if $n \leq 0$, return 0
- `sbfiz x3, x2, 2, 32` : $x3 = n * 4$ (byte size)
- `mov x2, 0` : loop index $i = 0$
- `ldr s2, [x0, x2]` : load $a[i]$
- `ldr s1, [x1, x2]` : load $b[i]$
- `add x2, x2, 4` : $i += 4$ bytes (1 float)
- `fmadd s0, s2, s1, s0` : $s0 = s0 + s2 * s1$
- `cmp x3, x2` : loop until $i == n * 4$

Results:

- Loads 1 float at a time
- Uses a loop
- Scalar multiply-accumulate (fmadd)

Manual NEON — Minimal Kernel (dot_product_neon.cpp)

```
.global      _Z18dot_product_neon_4PKfS0_
.type       _Z18dot_product_neon_4PKfS0_, %function
_Z18dot_product_neon_4PKfS0_:
```

```

.LFB6131:
    .cfi_startproc
    ldr    q0, [x0]
    ldr    q1, [x1]
    fmul   v1.4s, v1.4s, v0.4s
    dup    d0, v1.d[1]
    fadd   v0.2s, v0.2s, v1.2s
    dup    s1, v0.s[1]
    fadd   s0, s1, s0
    ret
    .cfi_endproc
.LFE6131:
    .size  _Zl8dot_product_neon_4PKfS0_, .-_Zl8dot_product_neon_4PKfS0_

```

A minimal NEON kernel operating on four floats, used to illustrate SIMD parallelism and manual reduction. The NEON function operates on a fixed-size input of 4 floats and is not a general replacement for the scalar implementation. The comparison focuses on instruction-level behavior and per-call overhead rather than full vector-length scalability.

- `ldr q0, [x0]` : load 4 floats from *a into q0 (v0)
- `ldr q1, [x1]` : load 4 floats from *b into q1 (v1)
- `fmul v1.4s, v1.4s, v0.4s` : $v1 = v1 * v0$ (4 parallel multiplications)
- `dup d0, v1.d[1]` : duplicate the upper 2 floats (as 64-bit) into d0
- `fadd v0.2s, v0.2s, v1.2s` : add lower 2 floats of v0 and v1 → partial reduction
- `dup s1, v0.s[1]` : duplicate the second float into s1
- `fadd s0, s1, s0` : final add → one float result

Results:

- Loads 4 floats at a time
- Multiplies all 4 in parallel
- Reduces them to a single float
- Fully vectorized and as a result faster
- Uses lane duplication (`dup`) and scalar `fadd` instructions to reduce a vector of four floats into a single scalar result.

Benchmark.cpp Timed Results

```
uchiha@raspberrypi:~/Documents/Neon $ ./benchmark
Initializing 100000000 elements...
Scalar Time: 0.0934 s (Result: 16777216.0000)
NEON Time:   0.0676 s (Result: 67108864.0000)
-----
Speedup Factor: 1.3823x
```

Figure 1. Screenshot of `benchmark.cpp` output for a single run

NEON Time shows that the intrinsic functions proved to be faster than the scalar approach. However, the speedup factor was lower than expected. Inspecting the assembly code from `benchmark.cpp` will give some insight on what exactly happened.

Compiler Auto-Vectorized Scalar (From `benchmark.cpp`)

```
ldr    q2, [x1, x3]
ldr    q1, [x0, x3]
fmul   v1.4s, v1.4s, v2.4s
dup    s3, v1.s[1]
fadd   s0, s0, s1
dup    s2, v1.s[2]
dup    s1, v1.s[3]
fadd   s0, s0, s3
fadd   s0, s0, s2
fadd   s0, s0, s1
```

Although written as a scalar loop in C++, the compiler automatically transformed the function into a NEON-vectorized implementation when compiled with `-O3`.

- `ldr q*` → vector loads
- `fmul v*.4s` → SIMD multiply
- `dup + fadd` → horizontal reduction

These instructions clearly show this is NEON code, not scalar, which is the reason it performed better than expected. However, the manual NEON code still performed slightly better. Inspecting the assembly code of the NEON function will again provide insight on why that is.

Manual NEON (Optimized in benchmark.cpp)

```
ldr    q1, [x0, x3]
ldr    q2, [x1, x3]
fmul   v1.4s, v1.4s, v2.4s
fadd   v0.4s, v0.4s, v1.4s
faddp  v0.4s, v0.4s, v0.4s
faddp  v0.4s, v0.4s, v0.4s
```

When integrated into the benchmark and compiled with -O3, the manual NEON implementation was further optimized by the compiler to support arbitrary input sizes, resulting in loop unrolling, vector accumulation, and scalar remainder handling.

- Vector accumulator (v0.4s)
- Pairwise reduction (faddp)
- Fewer instructions than compiler-generated scalar reduction
- Explains remaining performance advantage

```
fadd v0.4s, v0.4s, v1.4s
faddp v0.4s, v0.4s, v0.4s
```

These two instructions show why manual NEON still performed better:

- Vector accumulator
- Fewer scalar reductions
- Better instruction scheduling

Remaining elements are handled using scalar FMADD instructions: `fmadd s0, s2, s1, s0`

Summary Results

Table 2. Neon and Scalar Characteristic Concepts

Vectorized Neon Version	Scalar Version
Processes 4 floats at once	Processes 1 float per loop
No loop overhead	Loop overhead: compare, branch, indexing
Uses SIMD multiply and add	Scalar FMADD only
Maintains a vector accumulator and reduces at the end, minimizing reduction overhead	Loads + FMADD per iteration

This experiment demonstrates that modern compilers can generate efficient NEON code from scalar C++ loops. However, manually written NEON intrinsics still provide finer control over reduction strategy and instruction selection, allowing for modest performance gains over compiler-generated SIMD.